

Doc 10.2 — Arrow Functions & Default Parameters

এই ডকে আমরা JavaScript-এর সবচেয়ে বেশি ব্যবহৃত function syntax-এর একটি — **Arrow Function** (`=>`) — এবং function-এর **Default Parameters** শিখব।

বিশেষ করে React/Next.js code-এ arrow function অনেক বেশি দেখতে পাবে।

1. Arrow Function কী?

Arrow Function হলো function লেখার একটি shorter syntax।

Traditional function:

```
function add(a, b) {  
  return a + b;  
}
```

Arrow function:

```
const add = (a, b) => {  
  return a + b;  
};
```

দুটাই `a + b` return করছে।

2. Basic Arrow Function Syntax

```
const functionName = (parameters) => {  
  // function body  
};
```

Example:

```
const greet = (name) => {  
  console.log(`Hello ${name}`);  
};
```

```
greet("Shariar");
```

Output:

```
Hello Shariar
```

3. Arrow Function কেন ব্যবহার করি?

Arrow function-এর প্রধান সুবিধা:

- Shorter syntax
- Callback-এর জন্য খুব convenient
- Array methods-এর সাথে খুব সুন্দরভাবে কাজ করে
- React code-এ খুব common
- Lexical `this` behavior দেয়

তবে arrow function শুধু shorter syntax না—**regular function**-এর সাথে কিছু গুরুত্বপূর্ণ **semantic difference**-ও আছে, বিশেষ করে `this`, `arguments`, `new`, এবং `prototype` নিয়ে।

4. One Parameter হলে Parentheses Optional

একটি parameter থাকলে parentheses বাদ দেওয়া যায়।

```
const square = number => {  
  return number * number;  
};
```

এটা একই:

```
const square = (number) => {  
  return number * number;  
};
```

দুটাই valid।

Best Practice

Team/project-এর coding style অনুযায়ী consistent থাকো।

React code-এ দুটোই দেখতে পাবে:

```
const double = x => x * 2;
```

অথবা:

```
const double = (x) => x * 2;
```

5. Zero Parameter হলে Parentheses লাগবে

যদি কোনো parameter না থাকে:

```
const greet = () => {  
  console.log("Hello");  
};
```

এখানে `()` অবশ্যই লাগবে।

এটা invalid:

```
const greet = => {  
  console.log("Hello");  
};
```

6. Multiple Parameters হলে Parentheses লাগবে

```
const add = (a, b) => {  
  return a + b;  
};
```

এখানে:

```
a, b
```

দুইটি parameter।

7. Arrow Function-এর Explicit Return

Regular function-এর মতো arrow function-এও `return` লিখতে পারো।

```
const add = (a, b) => {  
  return a + b;  
};
```

এটাকে বলা যায় **explicit return**।

কারণ আমরা নিজেরা লিখেছি:

```
return a + b;
```

8. Implicit Return

Arrow function-এর সবচেয়ে useful feature-এর একটি হলো **implicit return**।

যদি function body-তে শুধু একটি expression return করতে হয়, তাহলে `{}` এবং `return` বাদ দিতে পারো।

```
const add = (a, b) => a + b;
```

এটা equivalent:

```
const add = (a, b) => {  
  return a + b;  
};
```

9. Implicit Return Mental Model

এই:

```
const square = (x) => x * x;
```

ভাবতে পারো:

```
const square = (x) => {  
  return x * x;  
};
```

অর্থাৎ:

```
=> expression  
  ↓  
implicit return
```

10. Multiple Statements হলে `{ }` ব্যবহার করতে হবে

যদি একাধিক statement থাকে:

```
const calculate = (a, b) => {  
  const result = a + b;  
  
  console.log(result);  
  
  return result;  
};
```

এখানে block body ব্যবহার করা হয়েছে।

কারণ একাধিক statement আছে।

11. Arrow Function — Expression vs Block Body

Expression body

```
const double = x => x * 2;
```

Implicit return।

Block body

```
const double = x => {  
  return x * 2;  
};
```

Explicit return ।

মনে রাখবে

```
=> expression  
→ implicit return  
  
=> { ... }  
→ explicit return দরকার, যদি value ফেরত দিতে চাও
```

12. Common Mistake — Block Body-তে return ভুলে

যাওয়া

এই code:

```
const add = (a, b) => {  
  a + b;  
};
```

এটা `a + b` return করবে না ।

Result:

```
console.log(add(10, 20));
```

Output:

```
undefined
```

সঠিক:

```
const add = (a, b) => {  
  return a + b;  
};
```

অথবা:

```
const add = (a, b) => a + b;
```

13. Returning an Object

এখানে beginner-দের একটা common confusion আছে।

তুমি যদি arrow function থেকে directly object return করতে চাও:

```
const createUser = () => ({  
  name: "Sharlar",  
  age: 31  
});
```

এখানে `()` ব্যবহার করা হয়েছে।

কারণ:

```
() => ({})
```

মানে object expression return করা।

14. কেন Object-এর Around `()` লাগে?

যদি লেখো:

```
const createUser = () => {  
  name: "Sharlar";  
};
```

JavaScript এটাকে object return হিসেবে interpret করবে না।

বরং `{}` function body হিসেবে ধরা হবে।

সঠিক:

```
const createUser = () => ({
  name: "Sharlar"
});
```

অথবা explicit return:

```
const createUser = () => {
  return {
    name: "Sharlar"
  };
};
```

15. Arrow Function with Conditional Expression

```
const getStatus = age =>
  age >= 18 ? "Adult" : "Minor";
```

Example:

```
console.log(getStatus(25));
```

Output:

```
Adult
```

এখানে ternary expression implicit return হচ্ছে।

16. Arrow Function with Template Literal

```
const greet = name => `Hello, ${name}`;
```

Example:

```
console.log(greet("Shariar"));
```

Output:

```
Hello, Shariar
```

17. Arrow Function with Array Methods

Arrow function-এর সবচেয়ে common use-এর একটি হলো array methods ।

```
const numbers = [1, 2, 3, 4];  
  
const doubled = numbers.map(number => number * 2);  
  
console.log(doubled);
```

Output:

```
[2, 4, 6, 8]
```

এখানে:

```
number => number * 2
```

একটি arrow function ।

এটা callback হিসেবে `map()` -এ পাঠানো হয়েছে ।

Callback আমরা পরে আলাদা করে বিস্তারিত শিখব ।

18. `filter()` Example

```
const numbers = [1, 2, 3, 4, 5, 6];  
  
const evenNumbers = numbers.filter(number => number % 2 === 0);
```

```
console.log(evenNumbers);
```

Output:

```
[2, 4, 6]
```

19. `find()` Example

```
const users = [  
  { id: 1, name: "Rahim" },  
  { id: 2, name: "Karim" },  
  { id: 3, name: "Shariar" }  
];  
  
const user = users.find(user => user.id === 3);  
  
console.log(user);
```

Output:

```
{ id: 3, name: "Shariar" }
```

20. Arrow Function vs Regular Function — Basic Syntax

Regular:

```
function add(a, b) {  
  return a + b;  
}
```

Arrow:

```
const add = (a, b) => a + b;
```

Arrow function ছোট এবং concise ।

কিন্তু দুটোকে সবক্ষেত্রে interchangeable ভাবা যাবে না ।

21. সবচেয়ে গুরুত্বপূর্ণ Difference — `this`

Arrow function-এর সবচেয়ে important semantic difference হলো:

Arrow function নিজের `this` তৈরি করে না ।

Arrow function surrounding/outer scope-এর `this` ব্যবহার করে ।

Regular function-এর নিজস্ব `this` behavior থাকে, যা call করার নিয়মের ওপর নির্ভর করে ।

Example:

```
const user = {  
  name: "Sharir",  
  
  regular: function () {  
    console.log(this.name);  
  },  
  
  arrow: () => {  
    console.log(this.name);  
  }  
};  
  
user.regular();  
user.arrow();
```

`regular` method-এ `this` সাধারণত `user` object-কে refer করবে ।

কিন্তু `arrow` function object method হিসেবে call হলেও নিজের `this` নেয় না ।

তাই arrow function-কে সাধারণত object method হিসেবে ব্যবহার করার সময় সাবধান হতে হয় ।

22. Lexical `this`

Arrow function-এর `this` হলো **lexically inherited** `this` ।

সহজভাবে:

Arrow function নিজের `this` decide করে না; যেখানে function তৈরি হয়েছে, সেই surrounding context-এর `this` ব্যবহার করে ।

এই behavior React class components, event handlers এবং nested callbacks-এর historical patterns বুঝতে useful ।

`this` আমরা পরে আলাদা section-এ আরও deepভাবে শিখব ।

23. Arrow Function-এর `arguments` নেই

Regular function-এর মধ্যে `arguments` object পাওয়া যায় ।

```
function test() {  
  console.log(arguments);  
}
```

কিন্তু arrow function-এর নিজের `arguments` object নেই ।

```
const test = () => {  
  console.log(arguments);  
};
```

এখানে `arguments` arrow function-এর নিজস্ব নয় ।

এই কারণেই variable number of arguments দরকার হলে modern JavaScript-এ **rest parameters** ব্যবহার করা হয় ।

এটা পরের Doc 10.3-এ বিস্তারিত আসবে ।

24. Arrow Function-এর `new` ব্যবহার করা যায় না

Regular function constructor হিসেবে ব্যবহার করা যেতে পারে:

```
function User(name) {  
  this.name = name;  
}  
  
const user = new User("Shariar");
```

Arrow function constructor নয়:

```
const User = name => {  
  this.name = name;  
};
```

এভাবে:

```
new User("Shariar");
```

করা যাবে না।

Arrow functions **constructors** নয়।

25. Arrow Function-এর নিজের prototype নেই

Regular constructor functions-এর prototype থাকে।

```
function User() {}  
  
console.log(User.prototype);
```

Arrow function:

```
const User = () => {};  
  
console.log(User.prototype);
```

Arrow functions-এর নিজস্ব prototype property থাকে না।

এটা constructor behavior-এর সাথে সম্পর্কিত।

26. কখন Arrow Function ব্যবহার করব?

Arrow function খুব useful:

- Short utility functions
- Array callbacks
- React event handlers
- React component functions
- `map`
- `filter`
- `find`
- `reduce`
- `setTimeout` callback
- Promise callbacks
- ছোট transformation functions

Example:

```
const double = number => number * 2;
```

27. কখন Regular Function ভালো?

যখন তোমার দরকার:

- নিজের `this` behavior
- Constructor function
- `arguments` object
- কিছু traditional function patterns
- Method semantics যেখানে dynamic `this` দরকার

Example:

```
const user = {  
  name: "Sharir",  
  
  greet() {  
    console.log(this.name);  
  }  
};
```

Object method-এর জন্য সাধারণত arrow function blindly ব্যবহার করা উচিত নয়।

28. Arrow Function — React Example

React-এ:

```
const handleClick = () => {  
  console.log("Button clicked");  
};
```

JSX:

```
<button onClick={handleClick}>  
  Click Me  
</button>
```

অথ inline:

```
<button onClick={() => console.log("Clicked")}>  
  Click Me  
</button>
```

React-এ arrow functions খুব common ।

29. Arrow Function — Data Transformation

API থেকে data:

```
const users = [  
  { name: "Rahim", age: 25 },  
  { name: "Karim", age: 30 },  
  { name: "Shariar", age: 31 }  
];
```

শুধু names:

```
const names = users.map(user => user.name);
```

Result:

```
["Rahim", "Karim", "Shariar"]
```

30. Default Parameters

এখন আসি দ্বিতীয় topic-এ।

Default parameter হলো function parameter-এর default value।

Example:

```
function greet(name = "Guest") {  
  return `Hello, ${name}`;  
}
```

Call:

```
console.log(greet());
```

Output:

```
Hello, Guest
```

31. কেন Default Parameter দরকার?

ধরো:

```
function greet(name) {  
  return `Hello, ${name}`;  
}  
  
console.log(greet());
```

Output:

```
Hello, undefined
```

Default parameter ব্যবহার করলে:

```
function greet(name = "Guest") {  
  return `Hello, ${name}`;  
}
```

এখন:

```
greet();
```

Output:

```
Hello, Guest
```

32. Basic Syntax

```
function functionName(parameter = defaultValue) {  
  // ...  
}
```

Example:

```
function calculate(price, tax = 10) {  
  return price + (price * tax) / 100;  
}
```

এখন:

```
calculate(100);
```

tax হবে 10 ।

33. Argument না দিলে Default Value

```
function greet(name = "Guest") {  
  return `Hello ${name}`;  
}  
  
console.log(greet());
```

Output:

```
Hello Guest
```

34. Argument দিলে Default Value ব্যবহার হবে না

```
function greet(name = "Guest") {  
  return `Hello ${name}`;  
}  
  
console.log(greet("Shariar"));
```

Output:

```
Hello Shariar
```

কারণ actual argument `"Shariar"` দেওয়া হয়েছে।

35. `undefined` দিলে কী হয়?

খুব important:

```
function greet(name = "Guest") {  
  return `Hello ${name}`;  
}  
  
console.log(greet(undefined));
```

Output:

Hello Guest

কারণ default parameter activate হয় যখন argument-এর value `undefined` হয়।

36. `null` দিলে **Default** কাজ করবে?

না।

```
function greet(name = "Guest") {  
  return `Hello ${name}`;  
}  
  
console.log(greet(null));
```

Output:

Hello null

কারণ:

```
undefined → default value  
null      → actual value
```

এটা খুব important।

37. Default Parameter + Multiple Parameters

```
function createUser(name, role = "user", active = true) {  
  return {  
    name,  
    role,  
    active  
  };  
}
```

Call:

```
console.log(createUser("Shariar"));
```

Result:

```
{  
  name: "Shariar",  
  role: "user",  
  active: true  
}
```

38. Default Parameter-এর Position

Default parameter সাধারণত শেষে রাখলে সহজে ব্যবহার করা যায়।

```
function createUser(name, role = "user") {  
  // ...  
}
```

এটা clear।

39. Default Parameter-এর আগে Required Parameter

```
function createUser(name, role = "user") {  
  return `${name} - ${role}`;  
}
```

Call:

```
createUser("Shariar");
```

Result:

```
Shariar - user
```

40. Default Parameter-এর পরে Parameter

JavaScript-এ technically possible:

```
function test(a = 10, b) {  
  return [a, b];  
}
```

Call:

```
test(undefined, 20);
```

Result:

```
[10, 20]
```

কিন্তু API design-এর দিক থেকে default parameters শেষে রাখলে code সাধারণত বেশি readable হয়।

41. Default Parameter vs `||`

অনেকে default value দেওয়ার জন্য `||` ব্যবহার করে:

```
const name = userName || "Guest";
```

কিন্তু `||` সব falsy value-কে fallback হিসেবে ধরে।

যেমন:

```
0  
""  
false  
null  
undefined  
NaN
```

সবগুলো falsy।

Default parameter-এর behavior আলাদা।

42. Default Parameter vs undefined

```
function test(value = 100) {  
  return value;  
}
```

No argument

```
test();
```

Result:

```
100
```

undefined

```
test(undefined);
```

Result:

```
100
```

null

```
test(null);
```

Result:

```
null
```

0

```
test(0);
```

Result:

0

false

```
test(false);
```

Result:

false

Empty string

```
test("");
```

Result:

""

43. Default Parameter vs Example

```
function test(value = 100) {  
  return value;  
}
```

```
test(0);
```

Result:

0

কিছু:

```
const value = 0 || 100;
```

Result:

100

কারণ 0 falsy।

44. Default Parameter vs ??

Nullish coalescing:

```
const value = input ?? 100;
```

এটা শুধু:

```
null  
undefined
```

এর জন্য fallback দেয়।

Default parameter-ও মূলত undefined argument-এর জন্য default apply করে।

তবে ?? এবং default parameter-এর exact evaluation context আলাদা।

45. Default Parameter Expression

Default value শুধু simple value হতে হবে এমন না।

Expression ব্যবহার করা যায়:

```
function greet(name = "Guest".toUpperCase()) {  
  return name;  
}
```

এখানে default expression evaluate হবে যখন parameter-এর value undefined হবে।

46. Default Parameter দিয়ে Function Call

Default value হিসেবে function call-ও করা যায়:

```
function getDefaultName() {  
  return "Guest";  
}  
  
function greet(name = getDefaultName()) {  
  return `Hello ${name}`;  
}
```

Call:

```
greet();
```

Result:

```
Hello Guest
```

47. Default Parameter-এর Evaluation

Default parameter প্রয়োজন হলেই evaluate হয়।

```
function getDefault() {  
  console.log("Default executed");  
  
  return 100;  
}  
  
function test(value = getDefault()) {  
  return value;  
}  
  
test(50);
```

এখানে default function call হবে না, কারণ `50` already দেওয়া হয়েছে।

কিন্তু:

```
test();
```

এবার:

```
Default executed
```

print হবে।

48. Default Parameter এবং Earlier Parameters

পরের default parameter আগের parameter ব্যবহার করতে পারে।

```
function multiply(a, b = a * 2) {  
  return a * b;  
}
```

Call:

```
multiply(5);
```

এখানে:

```
a = 5  
b = 10
```

Result:

```
50
```

49. Default Parameter এবং Scope

Default parameter-এর নিজের parameter scope আছে।

এখানে কিছু advanced scope behavior দেখা যায়:

```
function test(a = 10, b = a * 2) {  
  return b;  
}  
  
console.log(test());
```

Output:

```
20
```

কারণ `b` default expression-এ আগের parameter `a` available।

50. Arrow Function + Default Parameter

Arrow function-এর সাথেও default parameter ব্যবহার করা যায়।

```
const greet = (name = "Guest") => {  
  return `Hello ${name}`;  
};
```

আরও short:

```
const greet = (name = "Guest") => `Hello ${name}`;
```

51. Multiple Default Parameters + Arrow

```
const createUser = (  
  name = "Guest",  
  role = "user"  
) => ({  
  name,  
  role  
});
```

Example:

```
console.log(createUser());
```

Result:

```
{  
  name: "Guest",  
  role: "user"  
}
```

52. React Practical Example

ধরো reusable component function:

```
function getButtonText(text = "Submit") {  
  return text;  
}
```

এখন:

```
getButtonText();
```

Result:

```
Submit
```

আর:

```
getButtonText("Save");
```

Result:

```
Save
```

53. API Data Processing Example

```
const formatUser = (  
  name = "Unknown",  
  role = "user"  
) => {  
  return {  
    name,  
    role  
  };  
};  
  
console.log(formatUser("Shariar"));
```

Result:

```
{  
  name: "Shariar",  
  role: "user"  
}
```

54. Arrow Function + Default Parameter + Array Method

একসাথে:

```
const formatName = (name = "Guest") =>  
  name.trim().toUpperCase();  
  
const names = ["shariar", "rahim", "karim"];  
  
const formatted = names.map(name => formatName(name));  
  
console.log(formatted);
```

Result:

```
["SHARIAR", "RAHIM", "KARIM"]
```

এখানে কয়েকটি concept একসাথে কাজ করছে:

```
arrow function
+
default parameter
+
method callback
+
string methods
```

55. Common Mistake — Arrow Function-এর `{}`

এইটা:

```
const add = (a, b) => {
  a + b;
};
```

return করবে না।

সঠিক:

```
const add = (a, b) => a + b;
```

অথবা:

```
const add = (a, b) => {
  return a + b;
};
```

56. Common Mistake — Object Return

ভুল:

```
const getUser = () => {  
  name: "Sharlar"  
};
```

সঠিক:

```
const getUser = () => ({  
  name: "Sharlar"  
});
```

অথবা:

```
const getUser = () => {  
  return {  
    name: "Sharlar"  
  };  
};
```

57. Common Mistake — Default Parameter + `null`

```
function greet(name = "Guest") {  
  return name;  
}  
  
console.log(greet(null));
```

অনেকে ভাবতে পারে:

Guest

কিন্তু result:

null

কারণ default parameter শুধু `undefined`-এর ক্ষেত্রে activate হয়।

58. Common Mistake — Arrow Function সবসময় Regular Function-এর Replacement না

Arrow function:

```
const greet = () => {};
```

Regular function:

```
function greet() {}
```

দুটোর syntax আলাদা হওয়ার পাশাপাশি behavior-ও কিছু ক্ষেত্রে আলাদা।

বিশেষ করে:

- `this`
- `arguments`
- `new`
- `prototype`

এসব difference মনে রাখতে হবে।

59. Quick Comparison — Arrow vs Regular Function

Feature	Regular Function	Arrow Function
Short syntax	✗	✓
Implicit return	✗	✓
নিজের <code>this</code>	✓ dynamic <code>this</code> behavior	✗ lexical <code>this</code>
নিজের <code>arguments</code>	✓	✗
Constructor হিসেবে <code>new</code>	✓	✗
নিজের <code>prototype</code>	Constructor function হলে থাকে	✗
Callback-এর জন্য	ভালো	খুব convenient
React code	ব্যবহার হয়	খুব common

60. Quick Comparison — Return Syntax

Regular

```
function add(a, b) {  
  return a + b;  
}
```

Arrow explicit

```
const add = (a, b) => {  
  return a + b;  
};
```

Arrow implicit

```
const add = (a, b) => a + b;
```

তিনটিই একই result দিতে পারে।

61. Quick Comparison — Default Parameters

```
function greet(name = "Guest") {  
  return name;  
}
```

Input	Result
<code>greet()</code>	<code>"Guest"</code>
<code>greet(undefined)</code>	<code>"Guest"</code>
<code>greet(null)</code>	<code>null</code>
<code>greet("")</code>	<code>""</code>
<code>greet(0)</code>	<code>0</code>
<code>greet(false)</code>	<code>false</code>

62. Practice — Output Predict

Question 1

```
const add = (a, b) => a + b;  
  
console.log(add(10, 20));
```

Output কী?

Question 2

```
const square = x => x * x;  
  
console.log(square(5));
```

Output কী?

Question 3

```
const greet = (name = "Guest") => {  
  return `Hello ${name}`;  
};  
  
console.log(greet());  
console.log(greet("Sharlar"));
```

Output কী হবে?

Question 4

```
function test(value = 100) {  
  return value;  
}  
  
console.log(test());  
console.log(test(undefined));
```

```
console.log(test(null));  
console.log(test(0));
```

চারটি output কী হবে?

Question 5

```
const add = (a, b) => {  
  a + b;  
};  
  
console.log(add(10, 20));
```

কেন `undefined`?

63. Coding Challenge 1 — Arrow Function

একটি arrow function তৈরি করো:

```
double(10);
```

Expected:

```
20
```

64. Coding Challenge 2 — Even Number

Arrow function ব্যবহার করে:

```
isEven(8);
```

Expected:

```
true
```

65. Coding Challenge 3 — Default Parameter

এমন function তৈরি করো:

```
greet();
```

Expected:

```
Hello Guest
```

আর:

```
greet("Shariar");
```

Expected:

```
Hello Shariar
```

66. Coding Challenge 4 — Default Tax

Function:

```
calculatePrice(1000);
```

Default tax হবে।

আর:

```
calculatePrice(1000, 20);
```

এক্ষেত্রে tax হবে।

67. Coding Challenge 5 — Object Return

একটি arrow function তৈরি করো:

```
createUser("Sharlar", 31);
```

যেটা return করবে:

```
{  
  name: "Sharlar",  
  age: 31  
}
```

68. Interview Questions

Q1. Arrow function কী?

Function লেখার একটি concise syntax, যার নিজস্ব `this`, `arguments`, `prototype` এবং constructor behavior regular function-এর মতো নয়।

Q2. Arrow function-এর implicit return কী?

একটি expression body থাকলে `{}` এবং `return` বাদ দিয়ে সরাসরি expression লিখে value return করা।

```
const add = (a, b) => a + b;
```

Q3. Arrow function-এ `{}` ব্যবহার করলে কী হয়?

এটা block body হয়ে যায়। Value return করতে হলে explicit `return` লিখতে হয়।

Q4. Arrow function-এ object কীভাবে return করবে?

```
const getUser = () => ({  
  name: "Sharlar"  
});
```

Q5. Arrow function-এর `this` কেমন?

Arrow function নিজের `this` তৈরি করে না; surrounding lexical context-এর `this` ব্যবহার করে।

Q6. Arrow function-এর `arguments` object আছে?

নিজস্ব `arguments` object নেই।

Q7. Default parameter কখন activate হয়?

যখন corresponding argument `undefined` হয় বা argument দেওয়া হয় না।

Q8. `null` দিলে default parameter কাজ করে?

না।

```
function test(value = 10) {  
  return value;  
}  
  
test(null); // null
```

Q9. Arrow function কি constructor হিসেবে ব্যবহার করা যায়?

না।

```
new ArrowFunction();
```

ব্যবহার করা যায় না।

Q10. Arrow function কি regular function-এর complete replacement?

না। Syntax সহজ হলেও `this`, `arguments`, constructor behavior ইত্যাদিতে গুরুত্বপূর্ণ difference আছে।

69. Mental Model

Arrow function:

```
Regular Function
  ↓
Shorter Syntax
  ↓
Arrow Function
  ↓
=> expression
  ↓
Implicit Return
```

Default parameter:

```
Argument আছে?
|
├── value আছে
|   ↓
|   সেই value
|
└── undefined
    ↓
    Default value
```

70. Golden Rules ★

Rule 1

এক parameter হলে parentheses optional:

```
x => x * 2
```

Rule 2

Zero parameter হলে:

```
() => {}
```

Rule 3

Multiple parameter হলে:

```
(a, b) => {}
```

Rule 4

Expression body:

```
const add = (a, b) => a + b;
```

এখানে implicit return ।

Rule 5

Block body:

```
const add = (a, b) => {  
  return a + b;  
};
```

এখানে explicit return দরকার ।

Rule 6

Object directly return করতে:

```
() => ({  
  name: "Sharlar"  
})
```

Rule 7

Default parameter `undefined` -এর জন্য কাজ করে।

```
function test(value = 10) {}
```

Rule 8

`null`, `0`, `false`, `""` default parameter trigger করে না।

Rule 9

Arrow function-এর নিজের `this` নেই।

Rule 10

Arrow function-এর নিজের `arguments` নেই।

71. Final Mastery Checklist

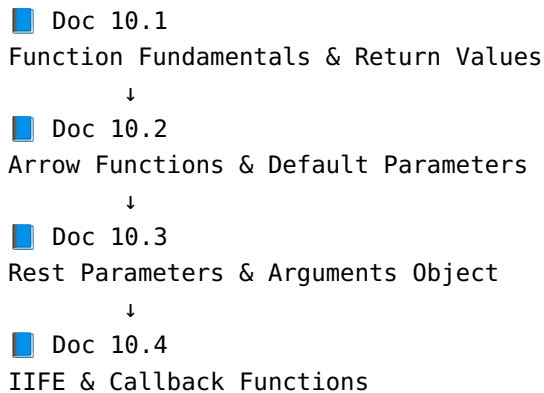
এই Doc শেষ করার পর তোমার confidently পারা উচিত:

- ☐ Arrow function কী
- ☐ Arrow function syntax
- ☐ Zero parameter
- ☐ One parameter
- ☐ Multiple parameters
- ☐ Explicit return
- ☐ Implicit return
- ☐ Expression body
- ☐ Block body
- ☐ Arrow function থেকে object return
- ☐ Arrow function with ternary
- ☐ Arrow function with template literals
- ☐ Arrow function with array methods
- ☐ Regular vs arrow function
- ☐ Arrow function-এর lexical `this`
- ☐ Arrow function-এর `arguments`
- ☐ Arrow function constructor না হওয়ার বিষয়
- ☐ Default parameters

- [] Default value
- [] `undefined` + default
- [] `null` + default
- [] `0` + default
- [] `false` + default
- [] Default parameter expression
- [] Multiple default parameters
- [] Default parameter vs `||`
- [] Default parameter vs `??`

Doc 10.2 Complete

এখন Functions section-এর flow:



পরের ডকে আমরা `...args` **Rest Parameters** + `arguments` **Object** নিয়ে যাব। এখানে বিশেষ করে `rest vs spread`, `extra arguments`, arrow function-এর `arguments` না থাকা—এসব খুব ভালোভাবে clear হবে।